МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Лабораторна робота № 1
з дисципліни «Сучасні інженерні мережеві технології»

Тема: Контейнерні мережі. Мережева модель Docker та взаємодія контейнеризованих сервісів

Виконав(ла): Прізвище_Ім'я, група ІО-00
Роботу прийняв: асистент кафедри ОТ Назар МЕЛЬНИК

Київ — 2026

1. Мета і завдання лабораторної роботи

Мета: ознайомитися з мережевою моделлю Docker, побудувати ізольовану користувацьку bridge-мережу, розгорнути в ній проксі, застосунок і базу даних, перевірити взаємодію, ізоляцію та публікацію портів, описати топологію у docker-compose.yml.

Завдання (високий рівень): основна частина (CLI-топологія nginx → Flask → Redis у мережі lab1-net, DNS, ізоляція, NAT, дослідження netns, Compose) та додаткова (сегментація frontend/backend, порівняння host/none/bridge, veth-пари, псевдоніми та балансування, healthcheck з depends_on).

2. Результати виконання

2.1. Схема спроєктованої мережевої топології

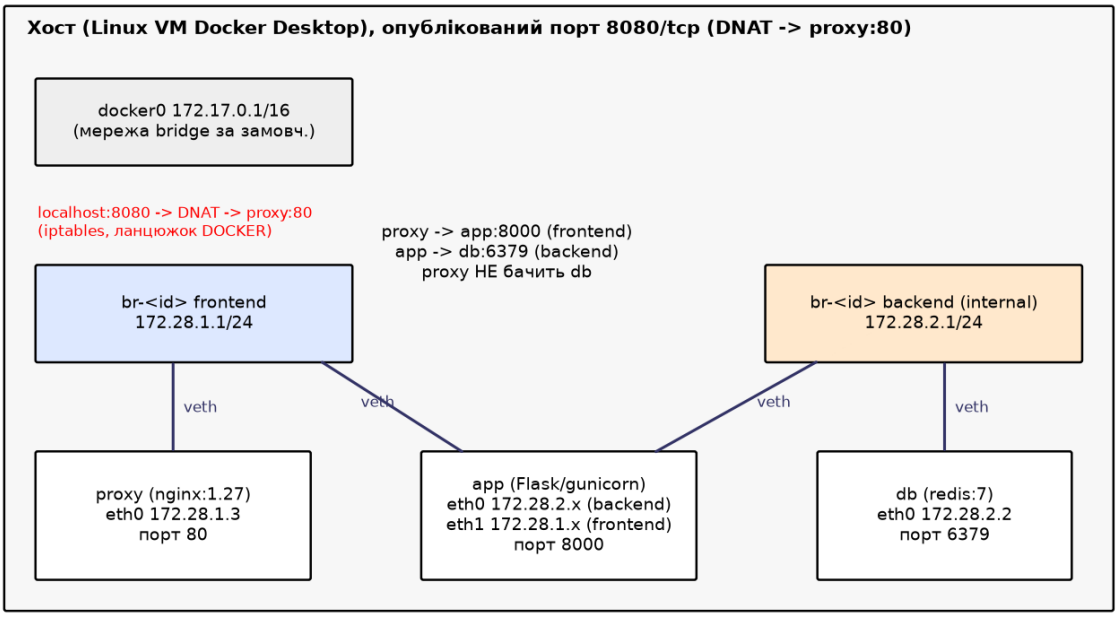


Рис. 1. Топологія: хост, мости, veth-пари, контейнери, IP-адреси, порти

Єдина точка входу — proxy (порт 8080 на хості). Застосунок app підключений до двох мереж: frontend (172.28.1.0/24) для зв'язку з проксі та backend (172.28.2.0/24, internal) для зв'язку з БД. У CLI-частині (основне завдання) усі три контейнери працюють в одній мережі lab1-net 172.28.0.0/24.

Сервіс	Образ	Порт	Мережі	Публікація
proxy	nginx:1.27-alpine	80	frontend	8080:80
app	lab1-app:1.0 (python:3.12-alpine, Flask, gunicorn)	8000	frontend, backend	ні
db	redis:7-alpine	6379	backend	ні

2.2. Середовище виконання

Версії

Команда:

```
docker version; docker compose version; docker info
```

Результат:

```
$ docker version --format 'Client {{.Client.Version}} / Server {{.Server.Version}} ({{.Server.O
  s}}/{{.Server.Arch}})'; docker compose version; docker info --format 'Driver={{.Driver}} Cgro
  upVersion={{.CgroupVersion}} Kernel={{.KernelVersion}} OS={{.OperatingSystem}}'
```

```
Client 28.5.2 / Server 28.5.2 (linux/arm64)
Docker Compose version v2.40.3-desktop.1
Driver=overlayfs CgroupVersion=2 Kernel=6.12.54-linuxkit OS=Docker Desktop
```

Інтерпретація: Використано Docker Engine 28.5.2 і Compose v2.40.3 на Docker Desktop (Linux VM, ядро 6.12, cgroup v2). Команди рівня хоста (ip, iptables, nsenter) виконуються у мережевому просторі цієї VM через привілейований контейнер.

2.3. Структура проєкту

- docker-compose.yml — опис топології (мережі, сервіси, healthcheck, depends_on);
- app/Dockerfile, app/app.py, app/requirements.txt — сервіс застосунку;
- nginx.conf — конфігурація проксі;
- run_cli.sh, cleanup_cli.sh — розгортання через Docker CLI (основне завдання);
- collect_evidence.sh — автоматичний збір виводів команд для звіту (evidence/).

2.4. Основне завдання

Пункт 2. Стандартні мережі Docker

Команда:

```
docker network ls; docker network inspect bridge
```

Результат:

```
$ docker network ls; docker network inspect bridge --format 'Name={{.Name}} Driver={{.Driver}}
  Subnet={{(index .IPAM.Config 0).Subnet}} Gateway={{(index .IPAM.Config 0).Gateway}}'
NETWORK ID     NAME      DRIVER    SCOPE
4c7577a2fac8   bridge    bridge    local
5ca8ab03fee6   host      host      local
c31edac7ad36   none      null      local
Name=bridge Driver=bridge Subnet=172.17.0.0/16 Gateway=172.17.0.1
```

Інтерпретація: Мережа bridge має підмережу 172.17.0.0/16, шлюз 172.17.0.1, драйвер bridge; поряд існують host і none.

Інтерфейс docker0 на хості

Команда:

```
ip -br addr show docker0; ip route | grep docker0
```

Результат:

```
$ hostexec 'ip -br addr show docker0; ip route | grep docker0'
docker0          UP             172.17.0.1/16 fe80::e094:d4ff:fed7:99fd/64
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
```

Інтерпретація: Інтерфейс docker0 має адресу шлюзу 172.17.0.1/16, а маршрут до 172.17.0.0/16 іде через цей міст.

Пункт 3. Користувацька мережа і контейнери

Команда:

```
bash run_cli.sh  (network create, run db/app/proxy)
```

Результат:

```
6fca28acdf61edf9c7f291efec55a98eb9b9d6dda6280a4310bf293c46a4ff6e
bb6f1a7718979c6fc77d46fa1bbe442054db9f2d59ac862dcd8bbe381615c63e
sha256:83c6d5c4fc083b7d4fdf9a4e03bb88ac0345bf2beff8d0255c967ba385beb61d
729a555d38d62a1094f6a774991d4c676506f37a1ebcfdf4c8103e81c507dedf
3353ab28db7cff8bb7e84757e1069806f202a3cfc25c167476a3dcb4709d8bef
{"hits":1,"instance":"729a555d38d6"}
```

Інтерпретація: Створено мережу lab1-net 172.28.0.0/24 і запущено db, app, proxy; відповідь {"hits":1,...} підтверджує роботу ланцюжка proxy → app → db.

Мережа lab1-net і міст br-<id>

Команда:

```
docker network inspect lab1-net; ip -br addr | grep br-
```

Результат:

```
$ docker network ls --filter name=lab1-net; docker network inspect lab1-net --format 'Subnet={{
```

```
  (index .IPAM.Config 0).Subnet}} Gateway={{(index .IPAM.Config 0).Gateway}} Driver={{.Driver}}
  '; hostexec "ip -br addr | grep -E '^br-'"
NETWORK ID     NAME       DRIVER    SCOPE
6fca28acdf61   lab1-net   bridge    local
Subnet=172.28.0.0/24 Gateway=172.28.0.1 Driver=bridge
br-6fca28acdf61  UP             172.28.0.1/24 fe80::48f9:18ff:fe59:d89f/64
```

Інтерпретація: Підмережа й шлюз збігаються із заданими, а на хості з'явився інтерфейс br-<id> з адресою 172.28.0.1/24.

Пункт 4. Взаємодія та вбудований DNS

Команда:

```
docker exec proxy wget -qO- http://app:8000/; docker exec app cat /etc/resolv.conf; dig app
```

Результат:

```
$ docker exec proxy wget -qO- http://app:8000/; docker exec proxy wget -qO- http://app:8000/; d
  ocker exec app cat /etc/resolv.conf; docker run --rm --network lab1-net nicolaka/netshoot dig
   +short app; docker run --rm --network lab1-net nicolaka/netshoot dig +short db
{"hits":2,"instance":"729a555d38d6"}
{"hits":3,"instance":"729a555d38d6"}
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.11
options ndots:0

# Based on host file: '/etc/resolv.conf' (internal resolver)
# ExtServers: [host(192.168.65.7)]
# Overrides: []
# Option ndots from: internal
172.28.0.3
172.28.0.2
```

Інтерпретація: Проксі звертається до app за іменем, лічильник зростає; у контейнері nameserver 127.0.0.11 — це вбудований DNS Docker; dig повертає IP-адреси app і db.

З'єднання app → db за іменем

Команда:

```
docker exec app python -c "gethostbyname('db')"; docker exec db redis-cli GET lab:hits
```

Результат:

```
$ docker exec app python -c "import socket;print('db ->',socket.gethostbyname('db'))"; docker e
  xec db redis-cli GET lab:hits
db -> 172.28.0.2
3
```

Інтерпретація: Ім'я db резолвиться в 172.28.0.2, а значення лічильника в Redis збігається з кількістю запитів — застосунок реально звертається до БД.

Пункт 5. Ізоляція

Команда:

```
docker run --network bridge ... ping app / ping <IP app> / curl
```

Результат:

```
$ echo 'IP app = 172.28.0.3'; docker exec iso ping -c 2 -W 1 app; docker exec iso ping -c 2 -W
  1 172.28.0.3; docker exec iso curl -s -m 3 http://172.28.0.3:8000/ || echo 'curl: недоступно
  (код '$?')'
IP app = 172.28.0.3
ping: app: Name does not resolve
PING 172.28.0.3 (172.28.0.3) 56(84) bytes of data.

--- 172.28.0.3 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1021ms

curl: недоступно (код 28)
```

Інтерпретація: Контейнер у мережі bridge не резолвить ім'я app і не досягає його за IP (100% втрат, тайм-аут curl): різні мережі ізольовані.

Пункт 6. Публікація портів і NAT

Команда:

```
docker port; curl localhost:8080/8000/6379; iptables -t nat -L DOCKER -n
```

Результат:

```
$ docker port proxy; docker port app || echo 'app: портів не опубліковано'; docker port db || e
  cho 'db: портів не опубліковано'; curl -s http://localhost:8080/; curl -s -m 3 http://localho
  st:8000/ || echo 'localhost:8000 недоступний'; curl -s -m 3 http://localhost:6379/ || echo 'l
  ocalhost:6379 недоступний'; hostexec 'iptables -t nat -L DOCKER -n'
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
{"hits":4,"instance":"729a555d38d6"}
localhost:8000 недоступний
localhost:6379 недоступний
Chain DOCKER (2 references)
target     prot opt source               destination
DNAT       6    --  0.0.0.0/0            0.0.0.0/0            tcp dpt:8080 to:172.28.0.4:80
```

Інтерпретація: Опубліковано лише 8080 → proxy:80; порти 8000 і 6379 з хоста недоступні. Правило DNAT перенаправляє tcp/8080 на 172.28.0.x:80.

Пункт 7. Мережевий стан зсередини контейнерів

Команда:

```
docker run --network container:<name> nicolaka/netshoot ip addr; ip route; ss -tlnp
```

Результат:

```
$ for c in proxy app db; do echo "=== $c"; docker run --rm --network container:$c nicolaka/nets
  hoot sh -c 'ip -br addr; ip route; ss -tlnp'; done
=== proxy
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if87        UP             172.28.0.4/24
default via 172.28.0.1 dev eth0
172.28.0.0/24 dev eth0 proto kernel scope link src 172.28.0.4
State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess
LISTEN 0      511          0.0.0.0:80         0.0.0.0:*
LISTEN 0      4096      127.0.0.11:39159      0.0.0.0:*
=== app
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if86        UP             172.28.0.3/24
default via 172.28.0.1 dev eth0
172.28.0.0/24 dev eth0 proto kernel scope link src 172.28.0.3
State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess
LISTEN 0      2048         0.0.0.0:8000       0.0.0.0:*
LISTEN 0      4096      127.0.0.11:34655      0.0.0.0:*
=== db
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if85        UP             172.28.0.2/24
default via 172.28.0.1 dev eth0
172.28.0.0/24 dev eth0 proto kernel scope link src 172.28.0.2
State  Recv-Q Send-Q Local Address:Port  Peer Address:PortProcess
LISTEN 0      4096      127.0.0.11:37337      0.0.0.0:*
LISTEN 0      511          0.0.0.0:6379       0.0.0.0:*
LISTEN 0      511             [::]:6379          [::]:*
```

Інтерпретація: Кожен контейнер має eth0 з адресою з lab1-net, маршрут за замовчуванням через 172.28.0.1 і слухає свій порт (80, 8000, 6379); збігається з docker network inspect.

Встановлені TCP-з'єднання

Команда:

```
ss -tn у просторах proxy, app, db
```

Результат:

```
$ for c in proxy app db; do echo "=== $c"; docker run --rm --network container:$c nicolaka/nets
  hoot ss -tn; done
=== proxy
State Recv-Q Send-Q Local Address:Port Peer Address:Port
=== app
State Recv-Q Send-Q Local Address:Port  Peer Address:Port
ESTAB 0      0         172.28.0.3:45368   172.28.0.2:6379
=== db
State Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTAB 0      0         172.28.0.2:6379   172.28.0.3:45368
```

Інтерпретація: Видно ESTABLISHED-з'єднання app ↔ db (порт 6379): це підтверджує, що застосунок підтримує з'єднання з БД.

Пункт 8. Compose з нуля

Команда:

```
docker compose up -d --wait; docker compose ps
```

Результат:

```
$ docker compose up -d --wait; docker compose ps
 Network lab1-backend  Creating
 Network lab1-backend  Created
 Network lab1-frontend  Creating
 Network lab1-frontend  Created
 Container db  Creating
 Container db  Created
 Container lab1-app-1  Creating
 Container lab1-app-1  Created
 Container proxy  Creating
 Container proxy  Created
 Container db  Starting
 Container db  Started
 Container db  Waiting
 Container db  Healthy
 Container lab1-app-1  Starting
 Container lab1-app-1  Started
 Container lab1-app-1  Waiting
 Container lab1-app-1  Healthy
 Container proxy  Starting
 Container proxy  Started
 Container db  Waiting
 Container lab1-app-1  Waiting
 Container proxy  Waiting
 Container lab1-app-1  Healthy
 Container proxy  Healthy
 Container db  Healthy
NAME         IMAGE              COMMAND                  SERVICE   CREATED          STATUS
... (скорочено, усього 35 рядків)
```

Інтерпретація: Усі три сервіси піднялися одним командою, db і app у стані healthy, опубліковано лише порт proxy.

Відтворюваність

Команда:

```
docker compose down -v; docker compose up -d --wait; curl localhost:8080
```

Результат:

```
$ docker compose down -v; docker compose up -d --wait; docker compose ps; curl -s http://localh
  ost:8080/; docker compose down -v
 Container proxy  Stopping
 Container proxy  Stopped
 Container proxy  Removing
 Container proxy  Removed
 Container lab1-app-1  Stopping
 Container lab1-app-1  Stopped
 Container lab1-app-1  Removing
 Container lab1-app-1  Removed
 Container db  Stopping
 Container db  Stopped
 Container db  Removing
 Container db  Removed
 Network lab1-frontend  Removing
 Network lab1-backend  Removing
 Network lab1-backend  Removed
 Network lab1-frontend  Removed
 Network lab1-frontend  Creating
 Network lab1-frontend  Created
 Network lab1-backend  Creating
 Network lab1-backend  Created
 Container db  Creating
 Container db  Created
 Container lab1-app-1  Creating
 Container lab1-app-1  Created
```

```
 Container proxy  Creating
 Container proxy  Created
 Container db  Starting
 Container db  Started
 Container db  Waiting
 Container db  Healthy
 Container lab1-app-1  Starting
 Container lab1-app-1  Started
 Container lab1-app-1  Waiting
 Container lab1-app-1  Healthy
 Container proxy  Starting
 Container proxy  Started
 Container proxy  Waiting
 Container db  Waiting
... (скорочено, усього 69 рядків)
```

Інтерпретація: Після повного видалення (-v) конфігурація відтворюється з нуля, застосунок відповідає.

Пункт 9 (схема топології) виконано на рис. 1. Висновок за схемою: bridge-міст виконує роль L2-комутатора, кожен контейнер приєднаний до нього через veth-пару, а вихід назовні та публікація портів реалізовані через NAT (iptables).

2.5. Додаткове завдання

Сегментація frontend/backend

Команда:

```
docker network inspect lab1-frontend lab1-backend
```

Результат:

```
$ docker network inspect lab1-frontend lab1-backend --format '{{.Name}} subnet={{(index .IPAM.C
  onfig 0).Subnet}} internal={{.Internal}} containers: {{range $id,$c := .Containers}}{{$c.Name
  }}={{$c.IPv4Address}} {{end}}'
lab1-frontend subnet=172.28.1.0/24 internal=false containers: proxy=172.28.1.3/24 lab1-app-1=17
  2.28.1.2/24
lab1-backend subnet=172.28.2.0/24 internal=true containers: lab1-app-1=172.28.2.3/24 db=172.28.
  2.2/24
```

Інтерпретація: У frontend є proxy та app, у backend — app та db (internal=true). Мережі мають різні підмережі.

Проксі не бачить БД, app бачить обидва сегменти

Команда:

```
wget db з proxy; ip addr у netns proxy та app; getent hosts
```

Результат:

```
$ docker exec proxy wget -qO- -T 2 http://db:6379 || echo 'proxy -> db: ім’я не резолвиться (із
  оляція)'; docker run --rm --network container:proxy nicolaka/netshoot ip -br addr; APP=$(dock
  er compose ps -q app); docker run --rm --network container:$APP nicolaka/netshoot ip -br addr
  ; docker run --rm --network container:$APP nicolaka/netshoot getent hosts proxy db
wget: bad address 'db:6379'
proxy -> db: ім’я не резолвиться (ізоляція)
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if112       UP             172.28.1.3/24
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if110       UP             172.28.2.3/24
eth1@if111       UP             172.28.1.2/24
172.28.1.3       proxy  proxy
172.28.2.2       db  db
```

Інтерпретація: Із proxy ім'я db не резолвиться; у proxy один інтерфейс (frontend), а в app два: eth0 (backend) і eth1 (frontend).

veth-пари

Команда:

```
cat /sys/class/net/eth0/iflink; ip link | grep veth
```

Результат:

```
$ for c in proxy app db; do echo "=== $c iflink=$(docker exec $c cat /sys/class/net/eth0/iflink
  )"; done; hostexec 'ip -o link | grep veth | cut -d: -f1,2'
=== proxy iflink=87
```

```
=== app iflink=86
=== db iflink=85
85: veth90ca0fa@if11
86: vethc58dbba@if11
87: veth9ee27ef@if11
94: veth2894b7e@if11
97: vethd94bc3e@if11
```

Інтерпретація: Значення iflink у контейнері (87 для proxy, 86 для app, 85 для db) дорівнює номеру veth-інтерфейсу на хості — так зіставляються кінці пари; на схемі це лінії «veth».

Псевдоніми та балансування

Команда:

```
docker compose up -d --scale app=2; dig +short app; curl localhost:8080 ×6
```

Результат:

```
$ docker compose up -d --scale app=2 --wait --no-recreate; docker compose ps app; docker run --
  rm --network lab1-frontend nicolaka/netshoot dig +short app; for i in 1 2 3 4 5 6; do curl -s
   http://localhost:8080/; done
 Container db  Running
 Container lab1-app-1  Running
 Container lab1-app-2  Creating
 Container lab1-app-2  Created
 Container proxy  Running
 Container db  Waiting
 Container db  Healthy
 Container lab1-app-2  Starting
 Container lab1-app-2  Started
 Container lab1-app-2  Waiting
 Container lab1-app-1  Waiting
 Container lab1-app-2  Healthy
 Container lab1-app-1  Healthy
 Container proxy  Waiting
 Container db  Waiting
 Container lab1-app-2  Waiting
 Container lab1-app-1  Waiting
 Container db  Healthy
 Container proxy  Healthy
 Container lab1-app-2  Healthy
 Container lab1-app-1  Healthy
NAME         IMAGE          COMMAND                  SERVICE   CREATED          STATUS
             PORTS
lab1-app-1   lab1-app:1.0   "gunicorn --bind 0.0…"   app       20 seconds ago   Up 14 seconds (
  healthy)   8000/tcp
lab1-app-2   lab1-app:1.0   "gunicorn --bind 0.0…"   app       7 seconds ago    Up 6 seconds (h
  ealthy)    8000/tcp
172.28.1.2
172.28.1.4
{"hits":1,"instance":"c1ce0dbb0c1d"}
{"hits":2,"instance":"58f4f4a527ba"}
{"hits":3,"instance":"c1ce0dbb0c1d"}
{"hits":4,"instance":"58f4f4a527ba"}
{"hits":5,"instance":"c1ce0dbb0c1d"}
{"hits":6,"instance":"58f4f4a527ba"}
```

Інтерпретація: DNS повертає дві адреси репліки app, а в відповідях з'являються різні instance. Розподіл нерівномірний: це DNS round-robin, а не балансувальник; nginx для цього перерезолвить ім'я (resolver 127.0.0.11).

Healthcheck і depends_on

Команда:

```
docker compose stop; docker compose up -d proxy
```

Результат:

```
$ docker inspect --format '{{.Name}} {{if .State.Health}}{{.State.Health.Status}}{{else}}no-hea
  lthcheck{{end}}' $(docker compose ps -q); docker compose stop db; docker compose ps -a --form
  at '{{.Service}} {{.Status}}'; docker compose stop app proxy db; docker compose ps -a --forma
  t '{{.Service}} {{.Status}}'; docker compose up -d proxy 2>&1 | grep -E 'Waiting|Healthy|Star
  ted'; sleep 2; docker compose ps -a --format '{{.Service}} {{.Status}}'
/db healthy
/lab1-app-1 healthy
```

```
/proxy no-healthcheck
 Container db  Stopping
 Container db  Stopped
db Exited (0) Less than a second ago
app Up 17 seconds (healthy)
proxy Up 11 seconds
 Container proxy  Stopping
 Container proxy  Stopped
 Container lab1-app-1  Stopping
 Container lab1-app-1  Stopped
 Container db  Stopping
 Container db  Stopped
db Exited (0) 1 second ago
app Exited (0) Less than a second ago
proxy Exited (0) Less than a second ago
 Container db  Started
 Container db  Waiting
 Container db  Healthy
 Container lab1-app-1  Started
 Container lab1-app-1  Waiting
 Container lab1-app-1  Healthy
 Container proxy  Started
db Up 13 seconds (healthy)
app Up 8 seconds (healthy)
proxy Up 2 seconds
```

Інтерпретація: Після зупинки всіх сервісів запуск proxy автоматично піднімає db, чекає healthy, потім app, і тільки потім proxy: працює depends_on з condition: service_healthy.

Порівняння мережевих режимів bridge, host, none

Один і той самий контейнер у трьох режимах

Команда:

```
docker run --network bridge|host|none nicolaka/netshoot ip addr; ping 1.1.1.1
```

Результат:

```
$ for m in bridge host none; do echo "=== --network $m"; docker run --rm --network $m nicolaka/
  netshoot sh -c 'ip -br addr; echo -n "internet: "; (ping -c1 -W2 1.1.1.1 >/dev/null 2>&1 && e
  cho OK) || echo FAIL'; done; docker run -d --rm --name pubtest --network none -p 9090:80 ngin
  x:1.27-alpine >/dev/null; echo "docker port (none): [$(docker port pubtest)]"; docker rm -f p
  ubtest >/dev/null 2>&1
=== --network bridge
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if101       UP             172.17.0.3/16
internet: OK
=== --network host
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             192.168.65.3/24 fdc4:f303:9324::3/64 fe80::3c4c:9bff:fe3a:6658/
  64
services1@if11   UP             192.168.65.6/32 fdc4:f303:9324::6/128 fe80::c6d:dbff:fe81:f2dc/
  64
docker0          UP             172.17.0.1/16 fe80::e094:d4ff:fed7:99fd/64
br-6fca28acdf61  UP             172.28.0.1/24 fe80::48f9:18ff:fe59:d89f/64
veth90ca0fa@if11 UP             fe80::8cef:cfff:fe61:9f1/64
vethc58dbba@if11 UP             fe80::f8a5:90ff:fe19:9a93/64
veth9ee27ef@if11 UP             fe80::c893:5aff:feb3:63ff/64
veth2894b7e@if11 UP             fe80::d491:98ff:fe67:e928/64
internet: OK
=== --network none
lo               UNKNOWN        127.0.0.1/8 ::1/128
internet: FAIL
docker port (none): []
```

Інтерпретація: У bridge є eth0 з адресою підмережі і вихід назовні; у host контейнер бачить інтерфейси хоста; у none лише lo і немає зовнішньої мережі.

Режим	Інтерфейси	IP-адреса	Доступ назовні	Публікація портів
bridge	lo, eth0 (veth)	з підмережі моста	так (NAT)	так (-p, DNAT)
host	усі інтерфейси хоста	адреса хоста	так	не потрібна (порт відкривається на хості)

Режим	Інтерфейси	IP-адреса	Доступ назовні	Публікація портів
none	тільки lo	немає	ні	ні (docker port порожній)

3. Конфігураційні файли та код

```
name: lab1

services:
  db:
    image: redis:7-alpine
    container_name: db
    # дані тільки в backend-мережі
    networks: [backend]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 3s
      timeout: 2s
      retries: 10
      start_period: 3s

  app:
    build: ./app
    image: lab1-app:1.0
    environment:
      REDIS_HOST: db
    # app бачить обидва сегменти
    networks:
      frontend:
      backend:
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).read()"
      interval: 3s
      timeout: 3s
      retries: 10
      start_period: 3s

  proxy:
    image: nginx:1.27-alpine
    container_name: proxy
    # єдиний опублікований порт
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on:
      app:
        condition: service_healthy

networks:
  frontend:
    name: lab1-frontend
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.1.0/24
          gateway: 172.28.1.1
  backend:
    name: lab1-backend
    driver: bridge
    # без виходу назовні
    internal: true
    ipam:
      config:
        - subnet: 172.28.2.0/24
          gateway: 172.28.2.1
```

docker-compose.yml

```
FROM python:3.12-alpine
WORKDIR /srv/app
# спочатку залежності, щоб кешувався шар
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
# запуск через gunicorn
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]
```

app/Dockerfile

```
import os
import socket

import redis
from flask import Flask, jsonify

app = Flask(__name__)

# підключення до БД за DNS-іменем сервісу
db = redis.Redis(
    host=os.environ.get("REDIS_HOST", "db"),
    port=6379,
    decode_responses=True,
    socket_connect_timeout=1,
    socket_timeout=1,
)


@app.get("/")
def index():
    try:
        # кожен запит збільшує лічильник у Redis
        hits = db.incr("lab:hits")
        return jsonify(hits=hits, instance=socket.gethostname())
    except redis.exceptions.RedisError:
        return jsonify(error="database unavailable"), 503


@app.get("/health")
def health():
    try:
        # перевірка зв'язку з БД для healthcheck
        db.ping()
        return jsonify(status="ok")
    except redis.exceptions.RedisError:
        return jsonify(error="database unavailable"), 503
```

app/app.py

```
server {
    listen 80;

    # вбудований DNS Docker, щоб бачити нові репліки app
    resolver 127.0.0.11 valid=2s;

    location / {
        # змінна змушує nginx резолвити ім'я на кожен запит
        set $upstream http://app:8000;
        proxy_pass $upstream;
        proxy_set_header Host $host;
    }
}
```

nginx.conf

```
#!/usr/bin/env bash
# Основне завдання ЛР1: топологія через Docker CLI в одній мережі lab1-net
set -euo pipefail
cd "$(dirname "$0")"

# виконання команди у мережевому просторі хоста (VM Docker Desktop)
hostexec() { docker run --rm --privileged --pid=host alpine nsenter -t 1 -m -u -n -i sh -c "$*"; }

docker network create --driver bridge --subnet 172.28.0.0/24 --gateway 172.28.0.1 lab1-net
docker run -d --name db --network lab1-net redis:7-alpine
```

```
docker build -q -t lab1-app:1.0 ./app
docker run -d --name app --network lab1-net -e REDIS_HOST=db lab1-app:1.0
docker run -d --name proxy --network lab1-net -p 8080:80 \
  -v "$PWD/nginx.conf:/etc/nginx/conf.d/default.conf:ro" nginx:1.27-alpine
sleep 3
curl -s http://localhost:8080/
```

run_cli.sh

4. Висновки

Побудовано ізольовану bridge-мережу з проксі, застосунком і БД; доведено роботу вбудованого DNS Docker, ізоляцію між мережами та публікацію лише одного порту через NAT. Дослідження netns, veth-пар і мосту br-<id> показало, що контейнерна мережа базується на Linux namespaces, veth і bridge. Сегментація frontend/backend ховає БД від проксі, healthcheck із depends_on задає правильний порядок запуску, а DNS-псевдонім дозволяє масштабувати app без зміни конфігурації. Топологія повністю відтворюється командою docker compose up -d.

Підпис виконавця: _____ Підпис викладача: _____